

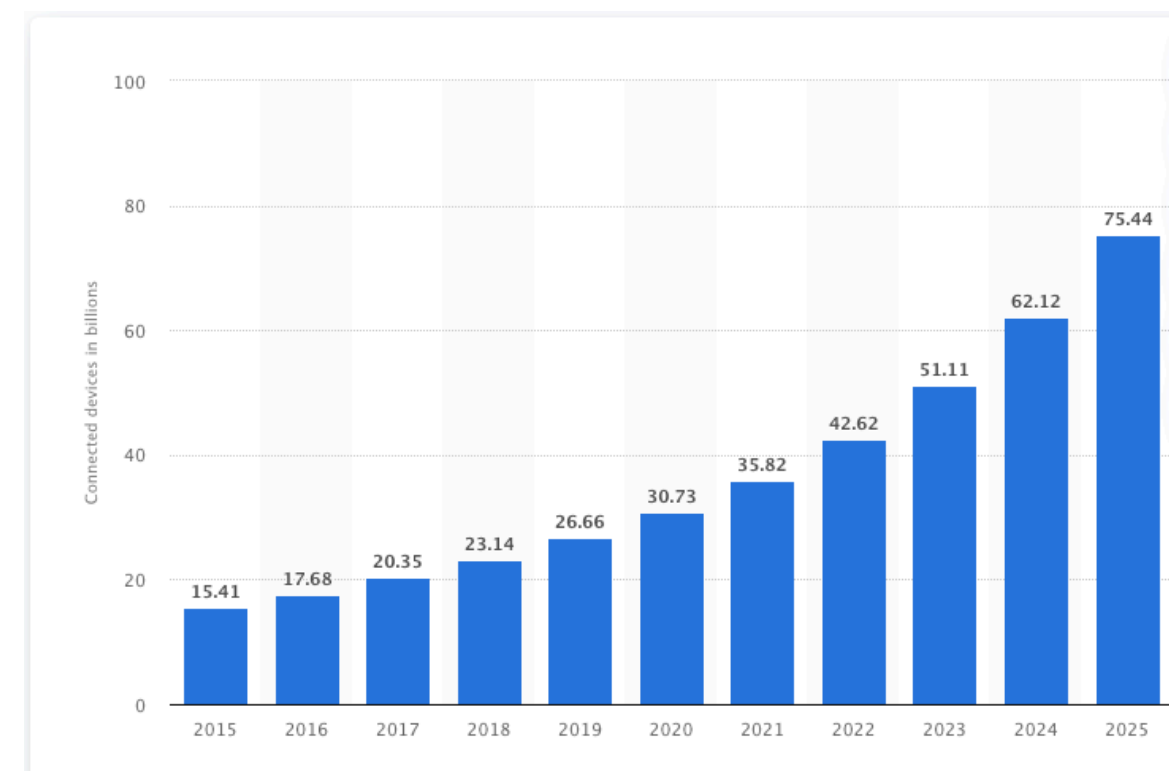
Software Dependability

Testing of Cyber-Physical Systems

Intro

Software is often embedded into complex environments, e.g., cars, robots, and missiles.

Embedded systems are "any sort of device which includes a programmable computer, but is not intended to be a general-purpose computer" - Marilyn Wolf.



In 2020, there were more than 30 billion embedded devices. The world industry spends more than 160 billion euros annually, with an annual growth rate of 9%.

It has been estimated that the average size of embedded systems is around one million lines of code, with notable outliers. A premium-class automobile is close to 100 million LOC, while a Boeing 787 Dreamliner is about 6.5 million LOC, only to operate onboard support systems and avionics.

Embedded systems affect major manufacturers and people's daily lives, such as battery charging, digital cameras, and video game consoles. The average middle-class American home has between 40 and 60 embedded processors.

Types of Embedded Systems

We can classify four embedded systems based on the services they offer.

General Computing. These are applications similar to desktop computing but run within embedded systems, e.g., video game consoles, wearable computers, and automatic tellers.

Control Systems. These applications perform closed-loop feedback control of real-time systems, e.g., vehicle engines, chemical processes, and nuclear power flight control.

Signal Processing. These applications help perform computations involving large data streams, e.g., radars, sonars, and video compression systems.

Communication and Networking. These applications revolve around switching and information transmissions, such as telephone systems and internet devices.

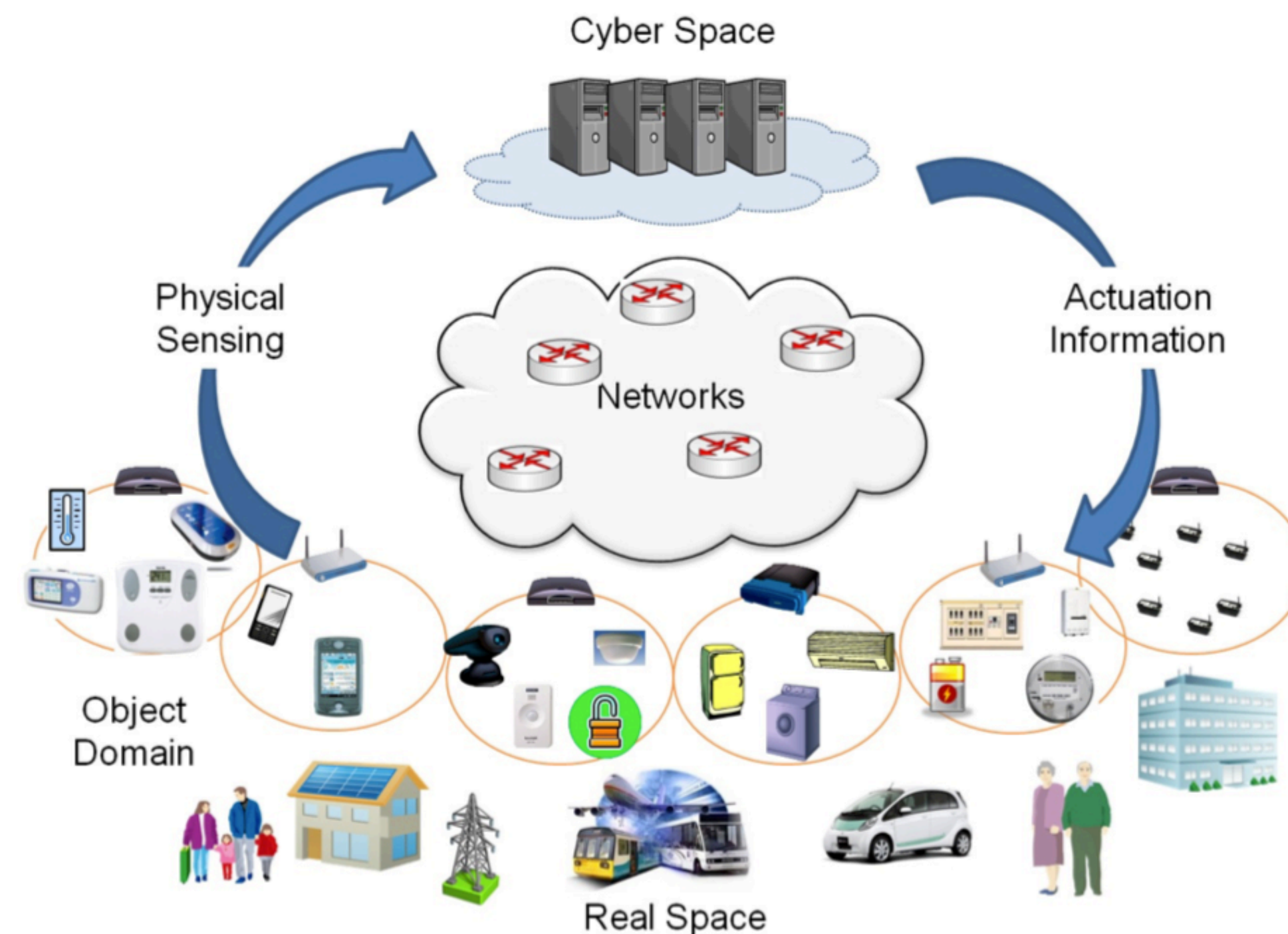


From Embedded to Cyber-Physical Systems

A cyber-physical system is a system of **collaborating computational elements controlling physical entities**. While embedded systems integrate information processing into products, cyber-physical systems orchestrate computational resources within physical systems and environments.

Embedded Systems + Physical Environment = Cyber-Physical System

e.g., smart electronic grids, air traffic management, and smart medical technologies.

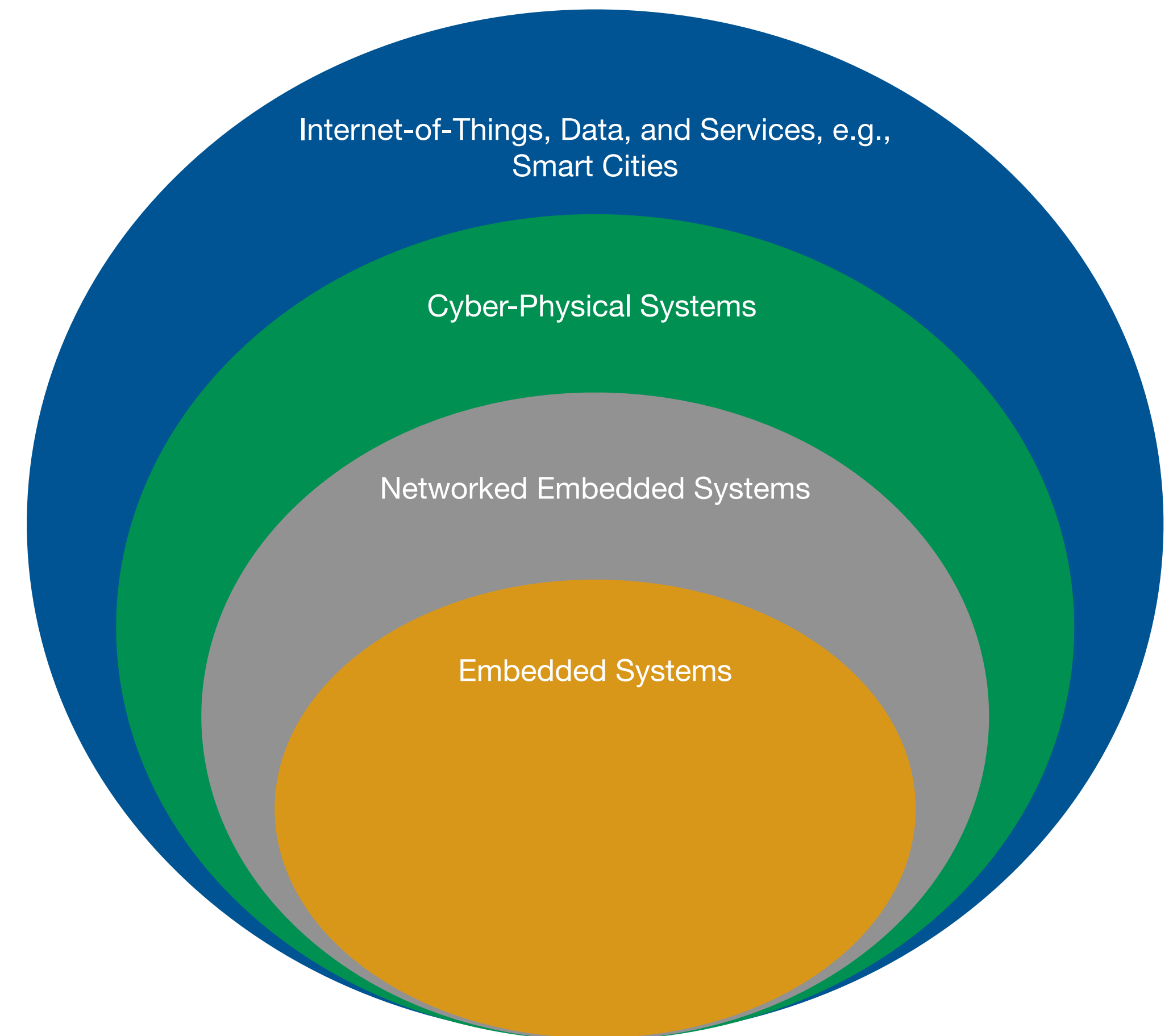


Cyber-Physical Systems practically enable the Internet-of-Things

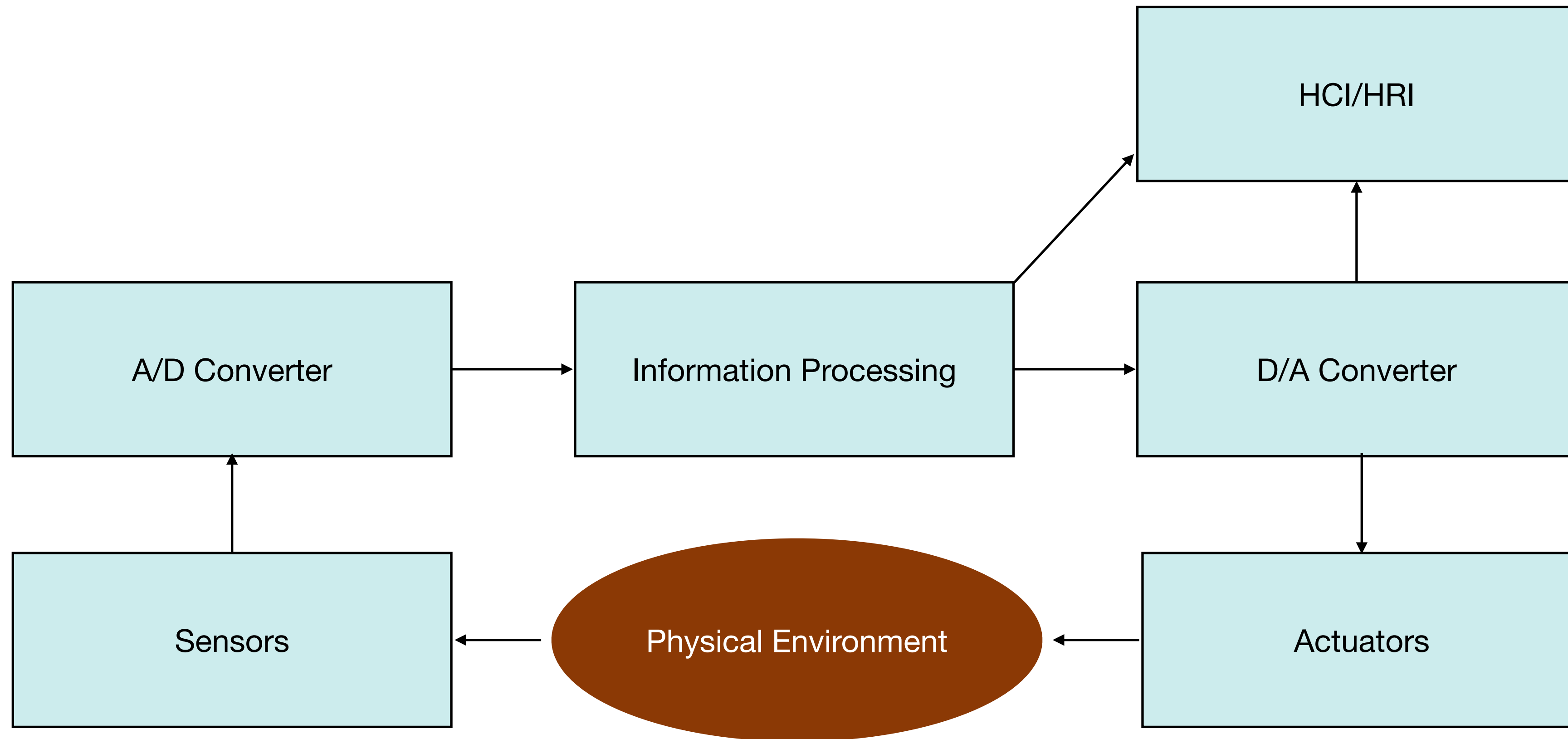
Emerging Cyber-Physical Systems combine embedded devices that communicate and may change their behavior depending on external conditions.

Such systems are **coordinated, distributed, and connected and must be robust and responsive.**

The **Internet of Things** aims to let computing devices and mechanical and digital machines collaborate to offer services.

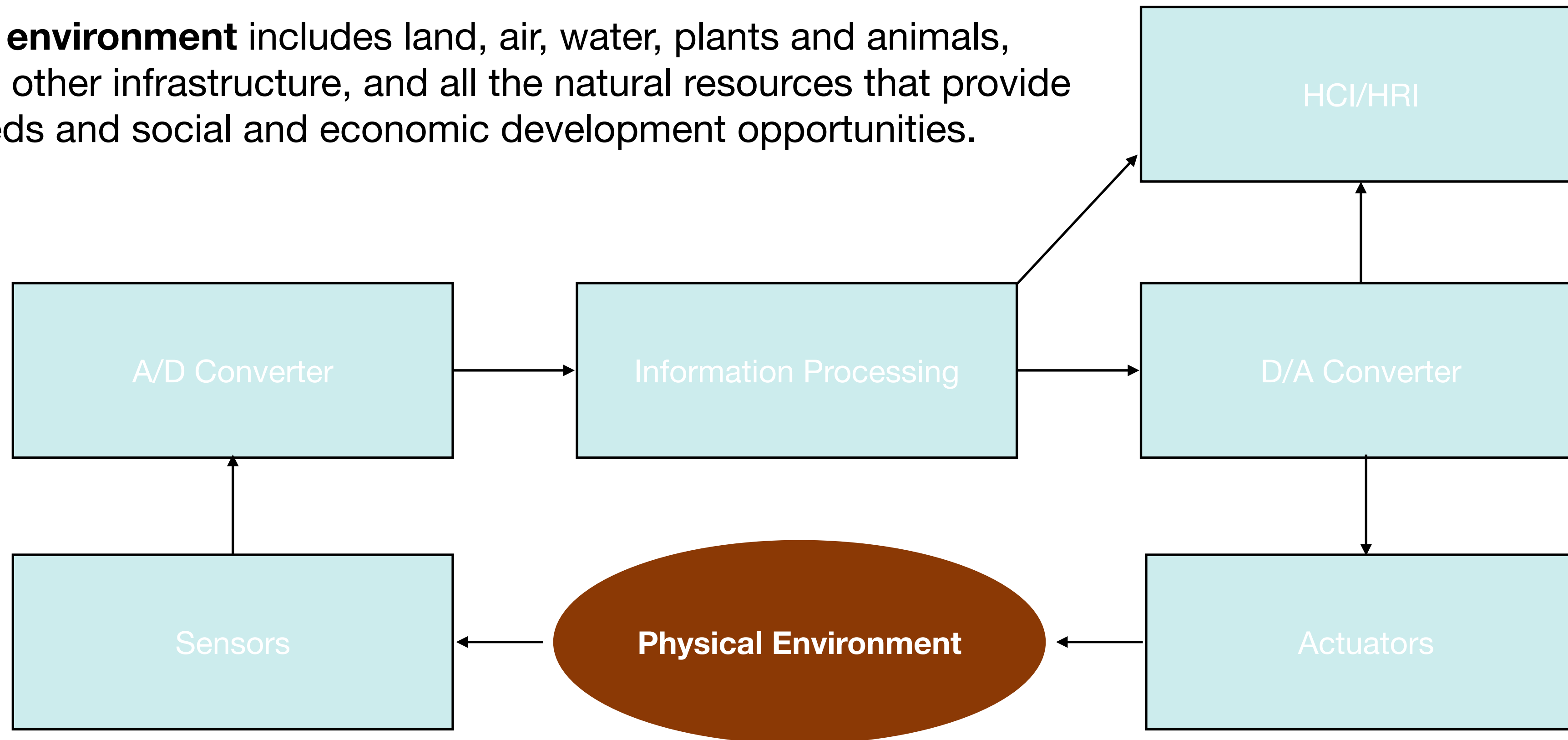


Components of a Cyber-Physical System



Components of a Cyber-Physical System

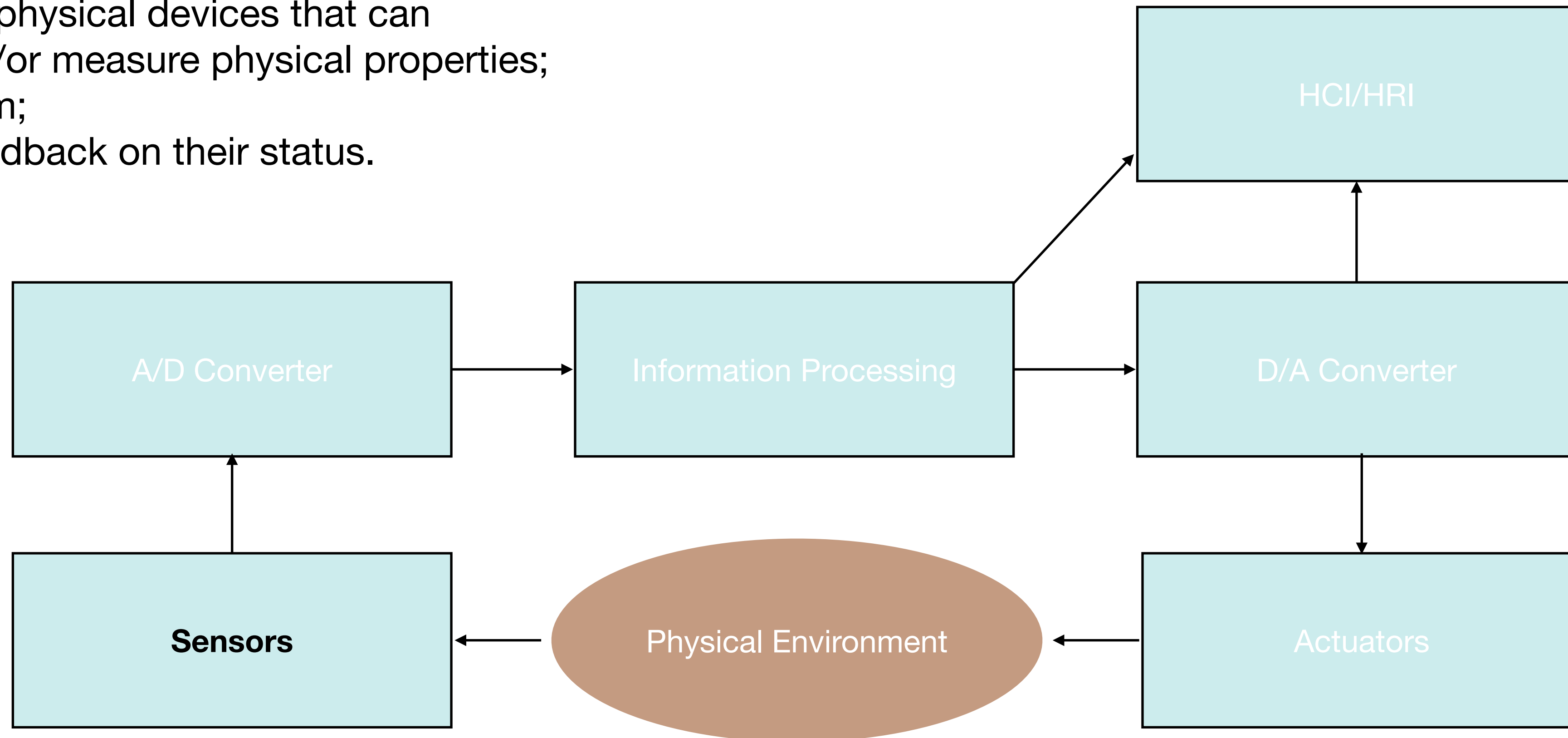
The **physical environment** includes land, air, water, plants and animals, buildings and other infrastructure, and all the natural resources that provide our basic needs and social and economic development opportunities.



Components of a Cyber-Physical System

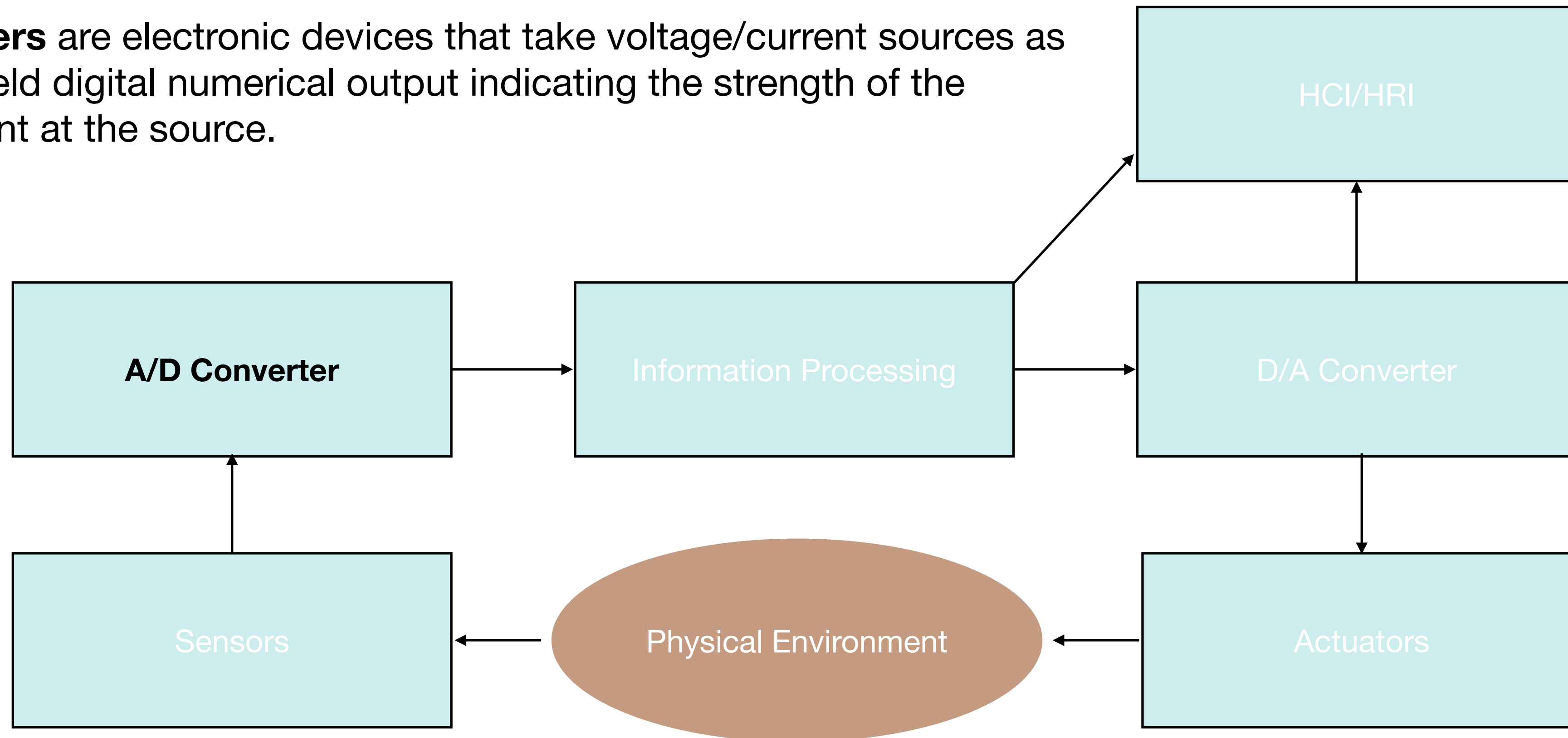
Sensors are physical devices that can

- detect and/or measure physical properties;
- record them;
- provide feedback on their status.



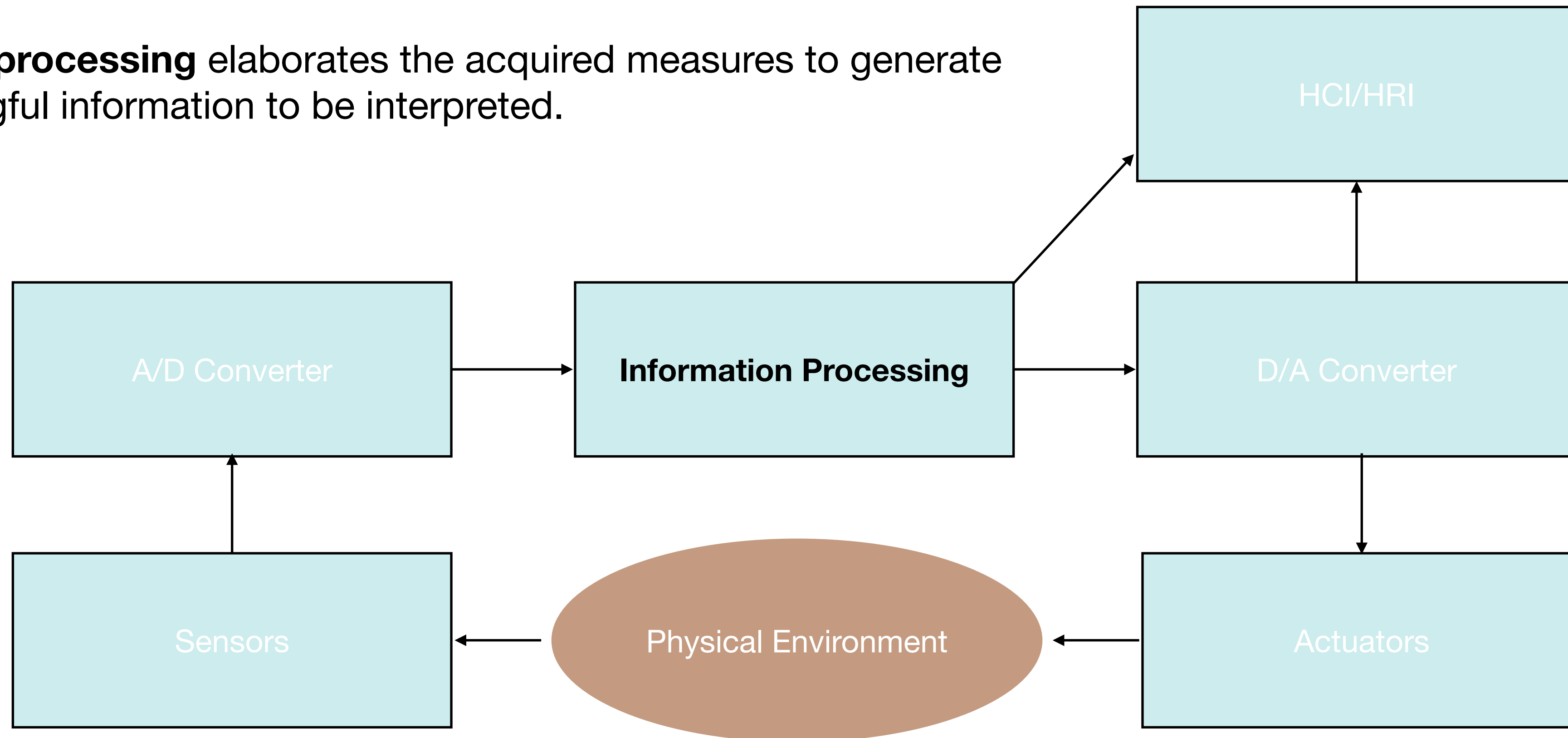
Components of a Cyber-Physical System

A/D converters are electronic devices that take voltage/current sources as inputs and yield digital numerical output indicating the strength of the voltage/current at the source.



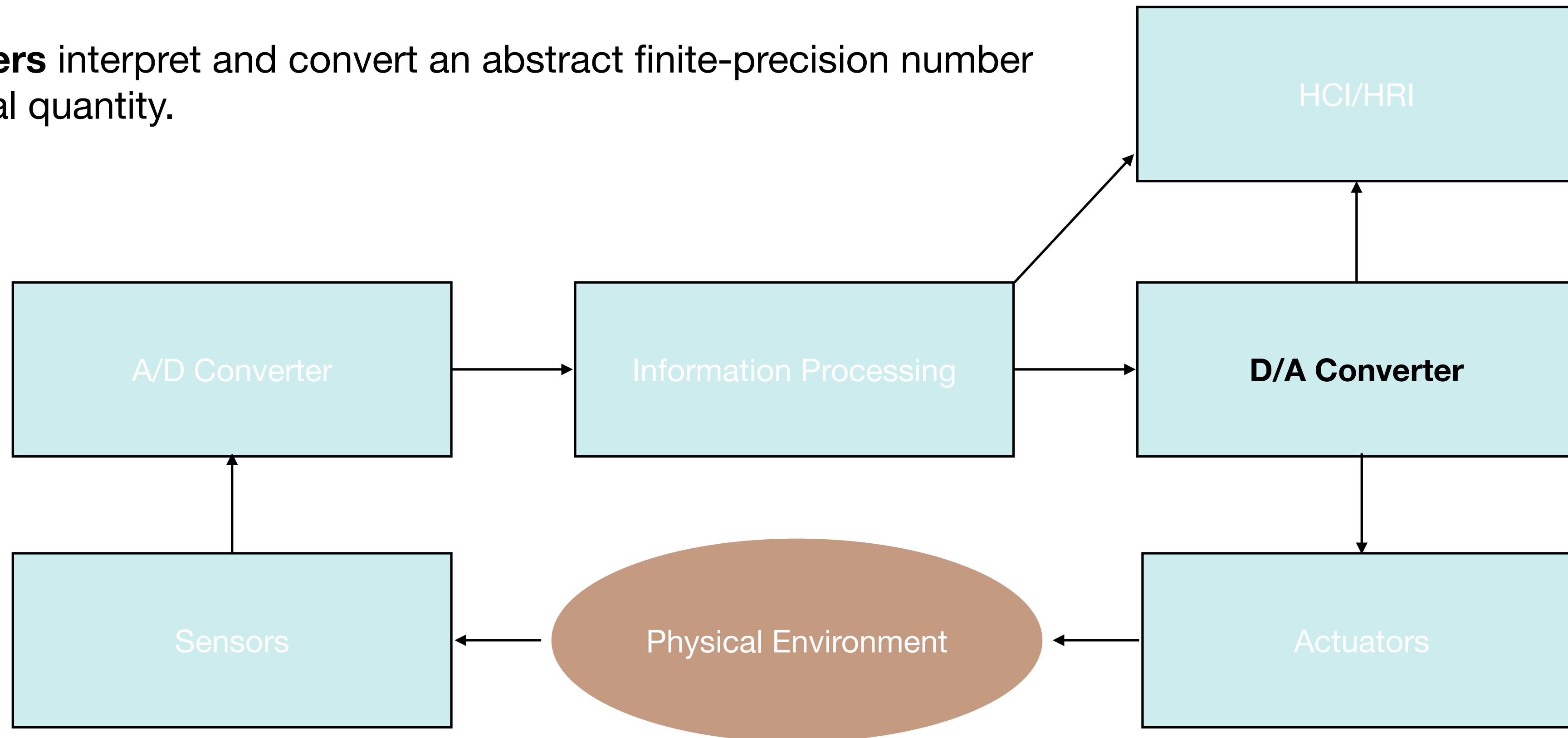
Components of a Cyber-Physical System

Information processing elaborates the acquired measures to generate new, meaningful information to be interpreted.



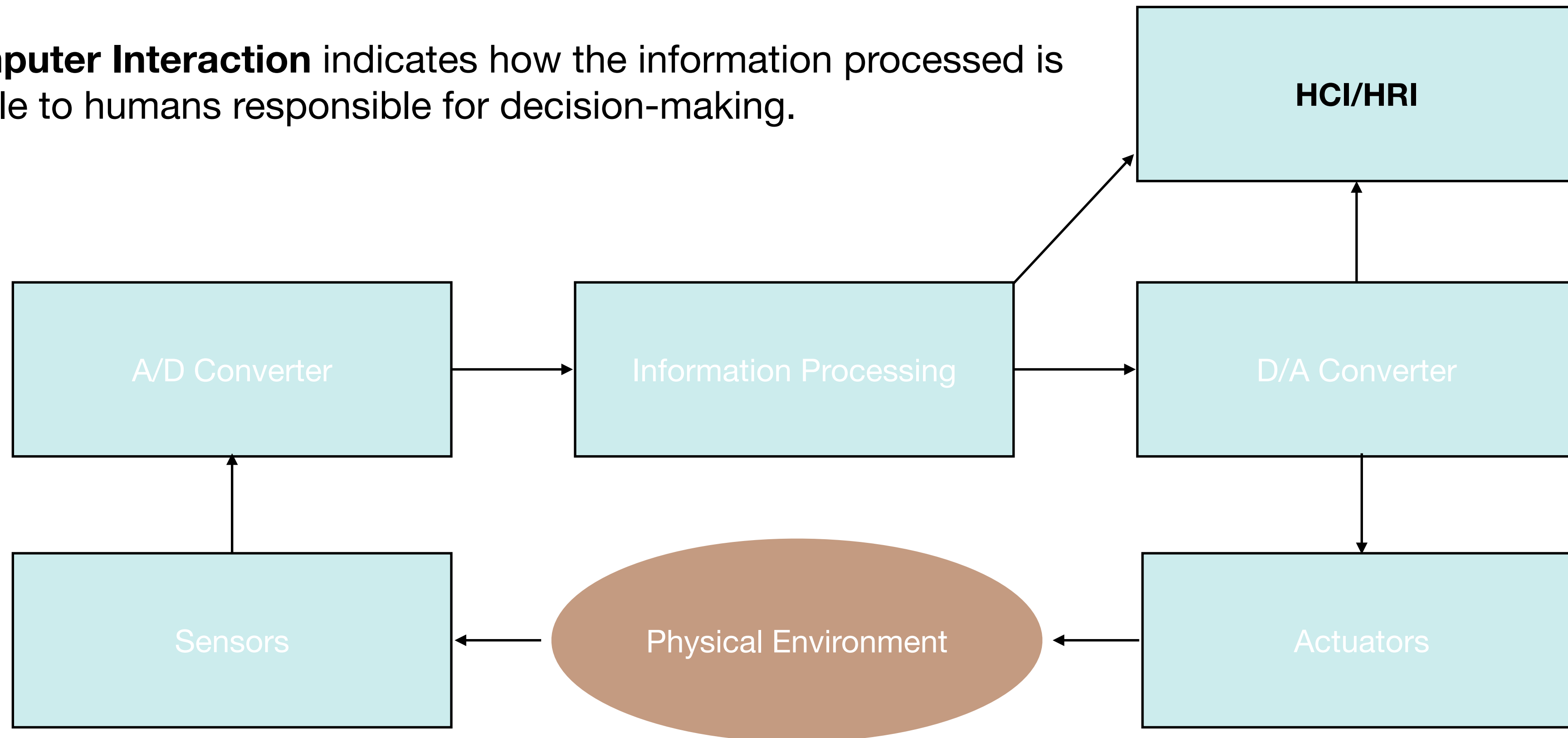
Components of a Cyber-Physical System

D/A converters interpret and convert an abstract finite-precision number into a physical quantity.



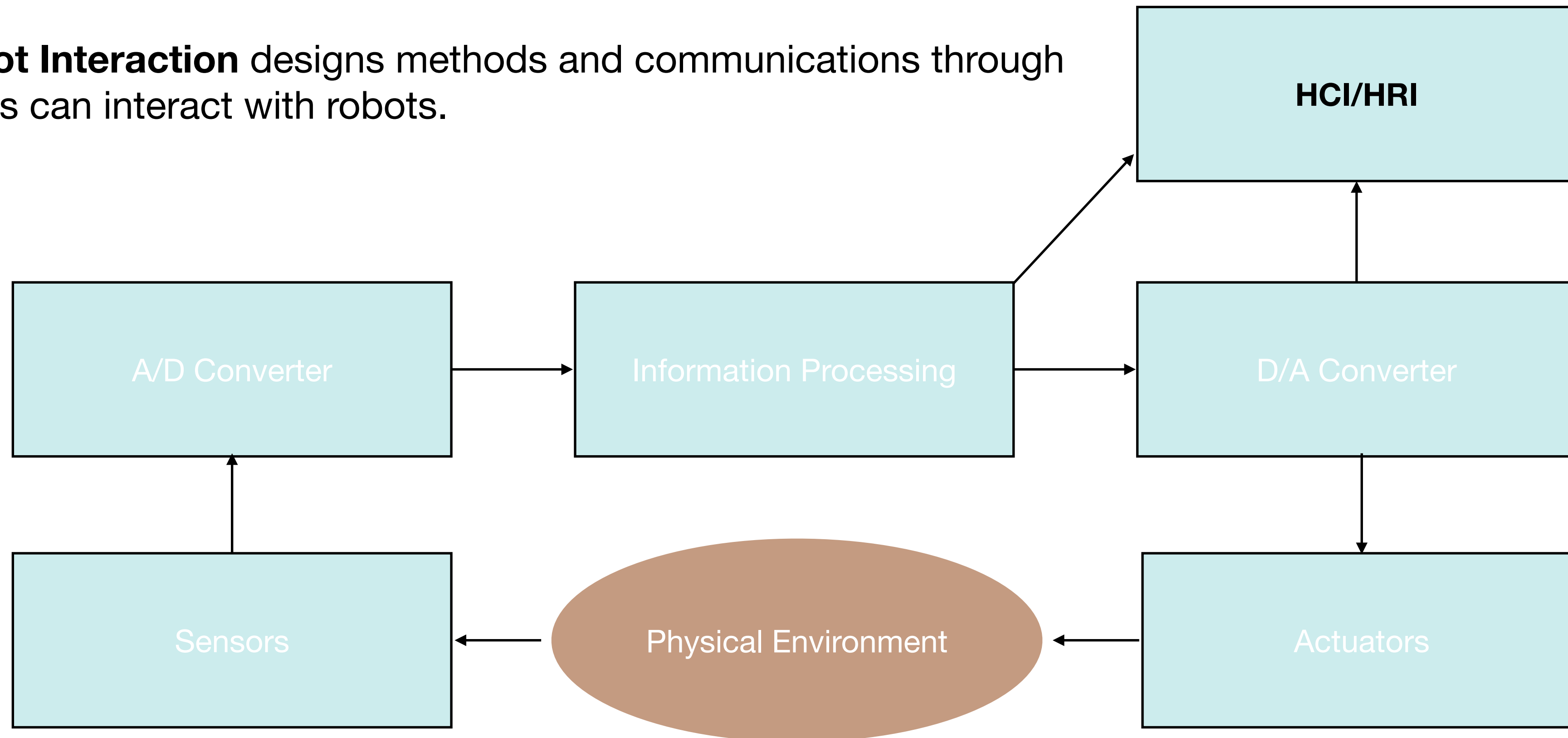
Components of a Cyber-Physical System

Human-Computer Interaction indicates how the information processed is made available to humans responsible for decision-making.



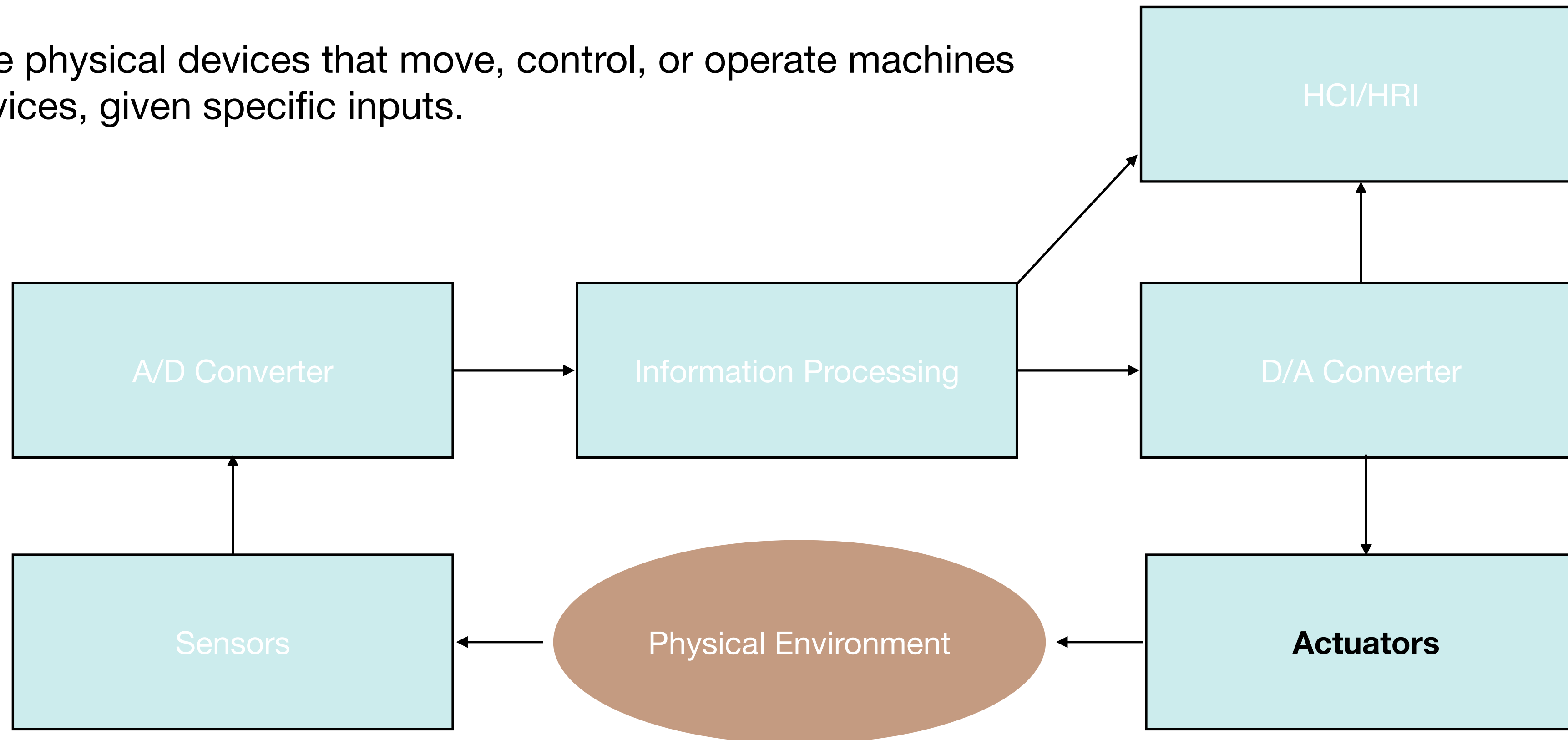
Components of a Cyber-Physical System

Human-Robot Interaction designs methods and communications through which humans can interact with robots.



Components of a Cyber-Physical System

Actuators are physical devices that move, control, or operate machines and other devices, given specific inputs.



Characteristics of Cyber-Physical Systems

Cyber-Physical Systems are **reactive** since they continuously interact with and react to the environment.

Cyber-Physical Systems are **hybrid** by nature and design, as they combine analog and digital components.

Cyber-Physical Systems are **dedicated** to specific applications: environmental knowledge is needed during design time to maximize robustness and efficiency.

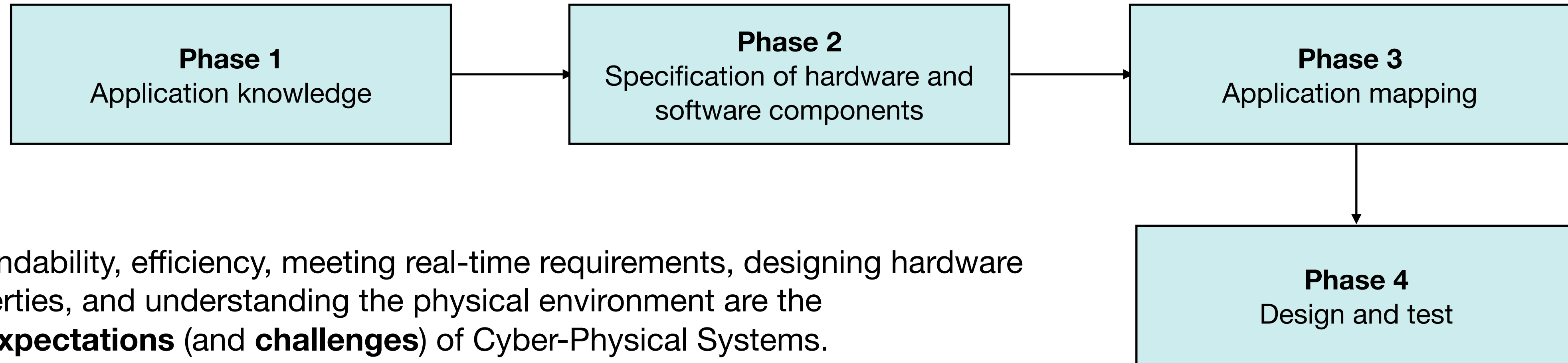
Cyber-Physical Systems are **dynamic** due to frequent environmental changes and the high volume and fluctuations of sensor data traffic.

Challenges of Cyber-Physical Systems

The characteristics of Cyber-Physical Systems increase the scientific and technical challenges across all phases of development, from design to testing.

Some examples are the complex integration of components, large-scale heterogeneous systems, interactions between humans and software, data uncertainty, and complex verification of system performance.

Designing a Cyber-Physical System



Dependability, efficiency, meeting real-time requirements, designing hardware properties, and understanding the physical environment are the key **expectations** (and **challenges**) of Cyber-Physical Systems.

If not meeting a **real-time constraint** could result in a catastrophe, it is called **hard**; all other real-time constraints are called **soft** [Kopetz, 1997].

Additional challenges concern **institutional** and **societal aspects**, e.g., trust, security, and data privacy.

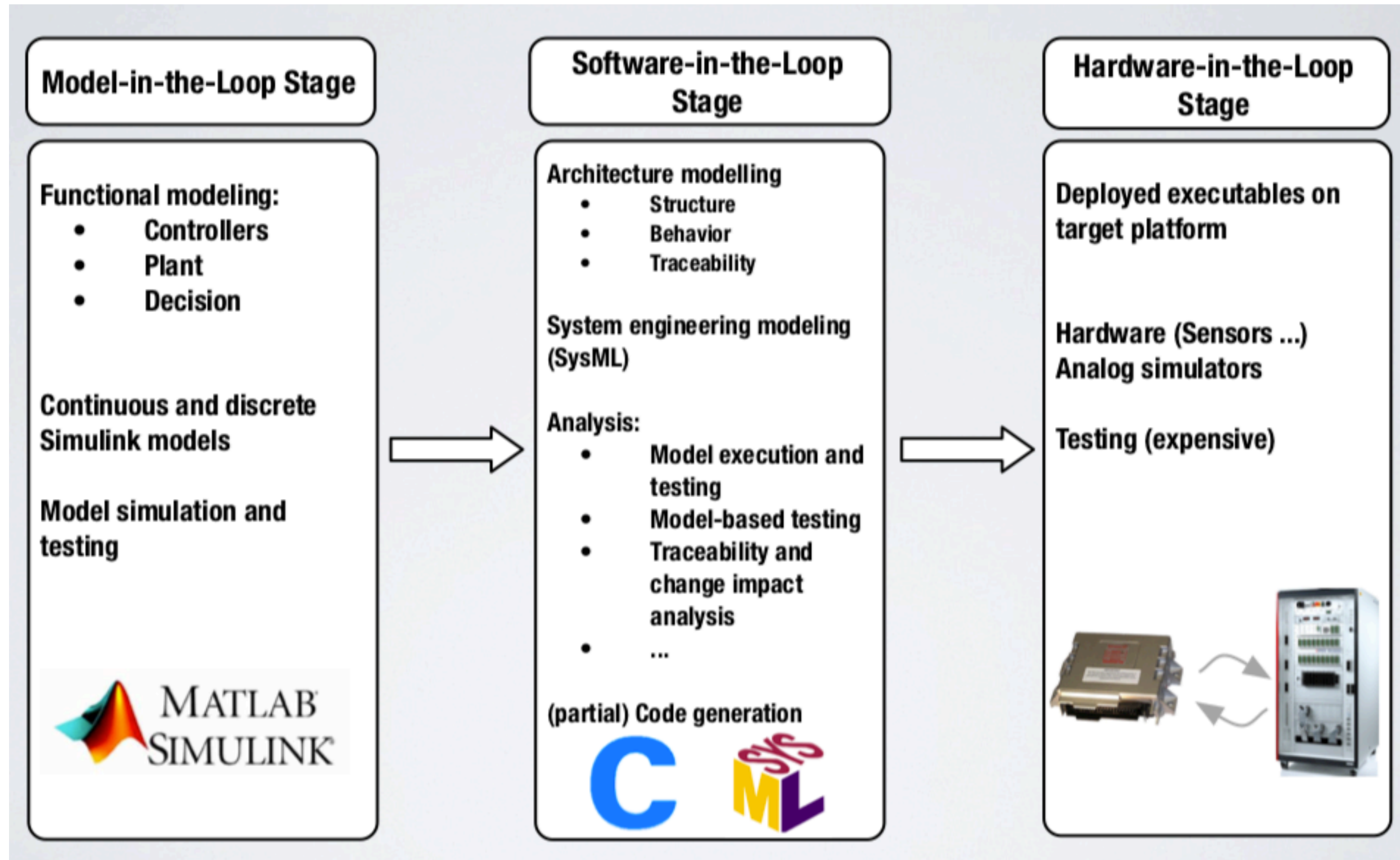
Data availability is particularly relevant: **real-time availability** provides a stricter operational environment than most IT systems.

These challenges imply a **stricter definition and usage of reliability engineering methods**.

Considerations about Testing Cyber-Physical Systems

- **Independent system testing.** We are naturally unaware of the system design and the surrounding environmental conditions.
- **Thorough knowledge of the application domain.** To perform testing, we must deeply understand the application domain and the typical use cases involving the Cyber-Physical System.
- **Environment simulation.** In *in-vitro conditions*, we cannot rely on the actual environment in which the system will operate; hence, we need to simulate it and the possible interacting phenomena.
- **Early testing.** Given the lack of instruments compared to other testing activities, it is even more critical to start testing early to improve the simulations and continuously reflect the working environment better.
- **Development-based testing.** The development process to create Cyber-Physical Systems is significantly different from that of traditional systems. These differences can be exploited to define tests at different granularities and at different times.

The Development Process of Cyber-Physical Systems



Model-in-the-Loop: Simulink

Simulink is a **visual programming interface** designed to make modeling systems intuitive.

It offers a way to solve equations numerically using a graphical user interface rather than code.

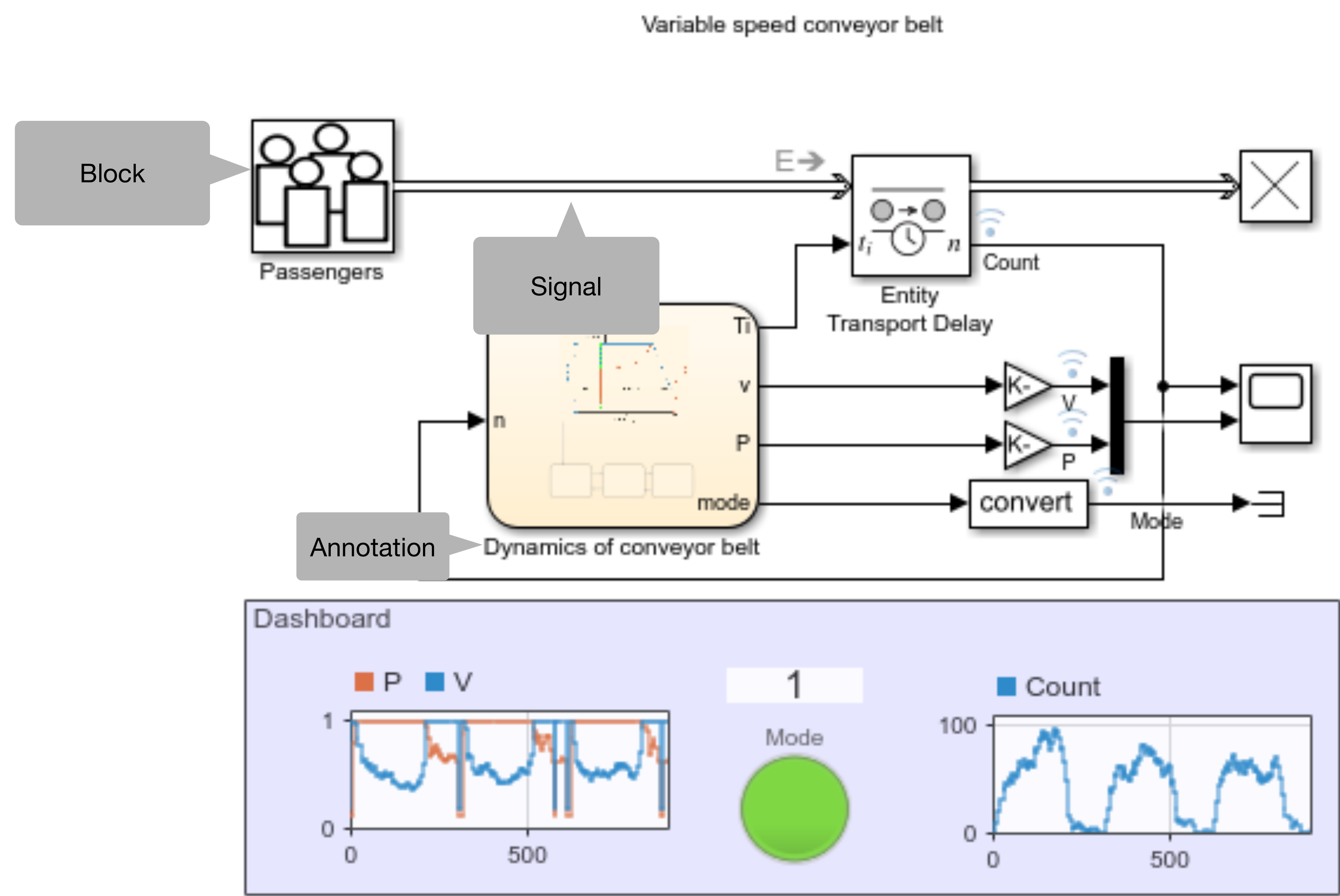
Models contain blocks, signals, and annotations:

- **Blocks** are mathematical functions. They can have varying numbers of inputs and outputs.
- **Signals** are lines that connect blocks and transfer values between them.
Signals have different data types, e.g., numbers, vectors, or matrices.
- **Annotations** of text or images can be added to the model.
While not used in the calculations, they can make it easier for others to understand design decisions in the model.

Typically, simulation models are used to design algorithms and compare different design options.

Cyber-Physical Systems simulation requires continuous-time, discrete-time, discrete-event, and finite-state modeling techniques. Let us consider an example...

Model-in-the-Loop: a Variable Speed Conveyor Belt System in Simulink



Model-in-the-Loop: a Variable Speed Conveyor Belt System in Simulink

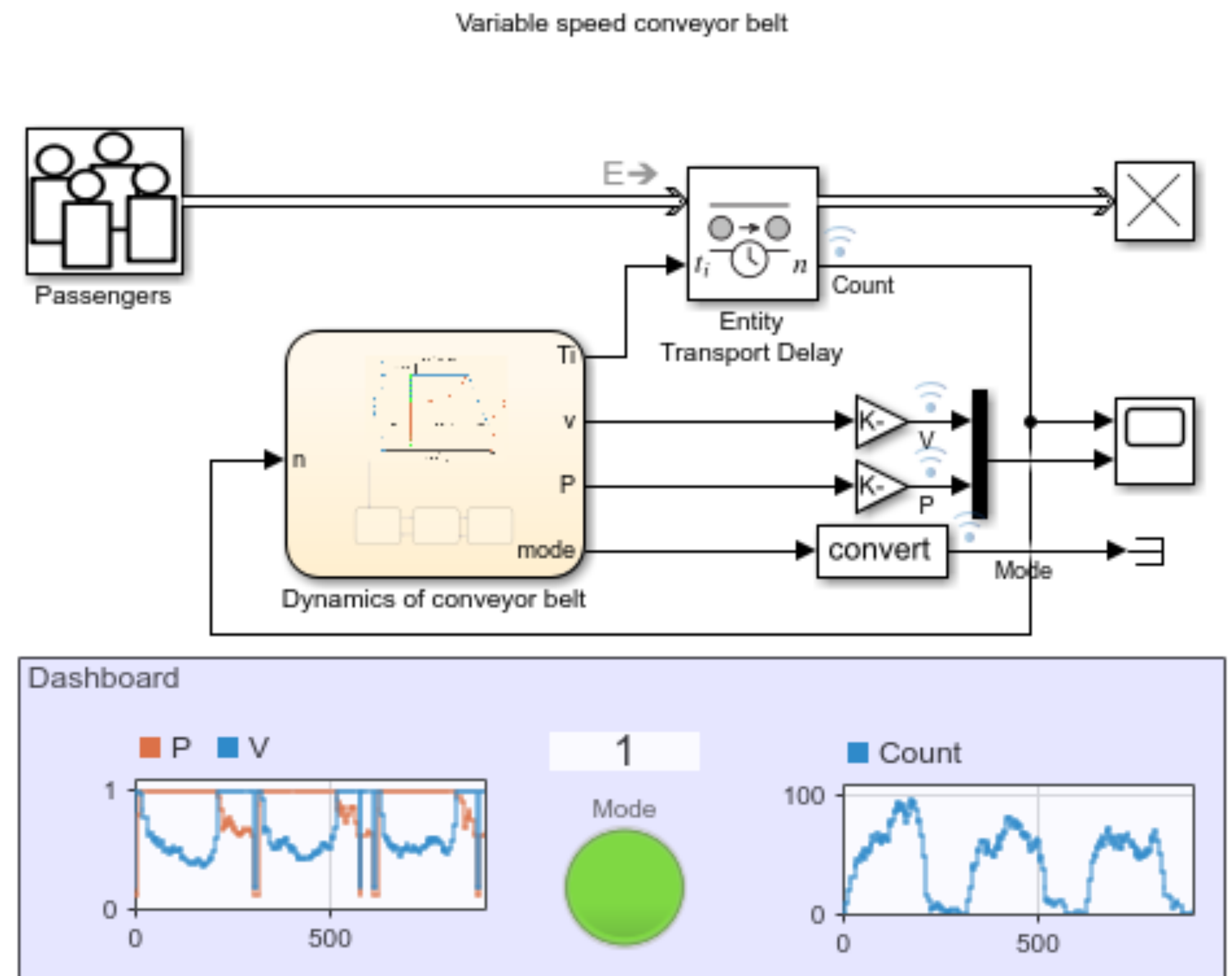
The 'Passengers' block models passenger arrivals as a Poisson process with exponential inter-arrival times, whose output is a sequence of entities representing the passengers who step onto the conveyor belt.

The inter-arrival time δ_t is exponentially distributed with the rate parameter λ following the function $f(\delta_t) = \lambda e^{-\lambda \delta_t}$ where λ is the arrival rate (passengers per unit time).

This behavior can be implemented using an *Entity Generator* block in Simulink, which generates entities based on the specified inter-arrival time distribution.

In the example, λ varies with time to represent three scenarios:

$$\lambda(t) = \begin{cases} 2 & \text{if } \text{mod}(t,300) \in [0,180], \text{ rush hour} \\ 0.5 & \text{if } \text{mod}(t,300) \in [180,240], \text{ normal hour} \\ 0.1 & \text{if } \text{mod}(t,300) \in [240,300], \text{ free hour} \end{cases}$$



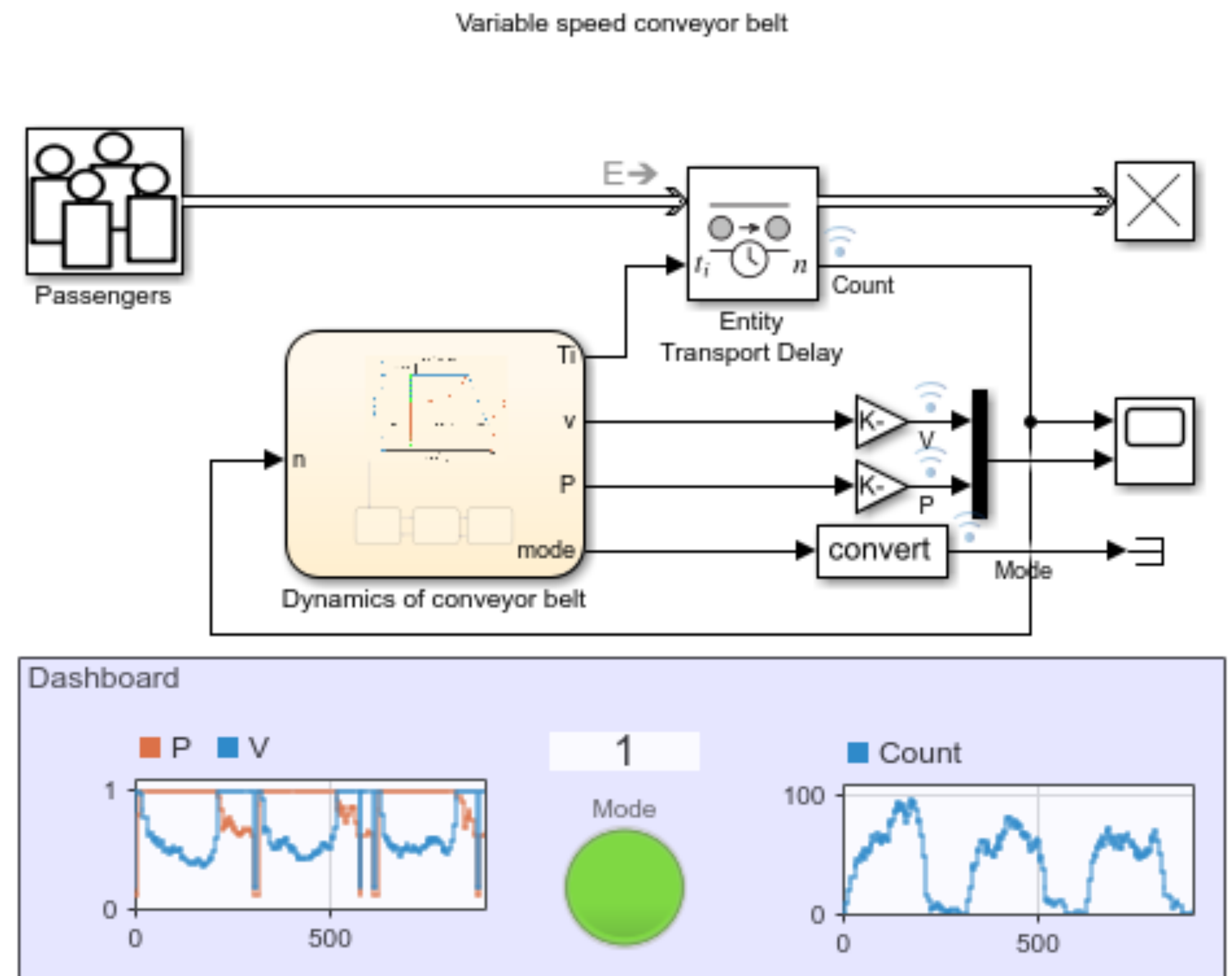
Copyright 2018-2019 The MathWorks, Inc.

Model-in-the-Loop: a Variable Speed Conveyor Belt System in Simulink

The 'Entity Transport Delay' block holds the passengers on the conveyor belt until they arrive at the other terminal, based on the time delay calculated by the Stateflow chart.

The 'Dynamics of conveyor belt' block models a variable-speed conveyor belt.

The 'Dashboard' will display the outcome of the simulations based on the load and reaction of the variable-speed conveyor belt system.



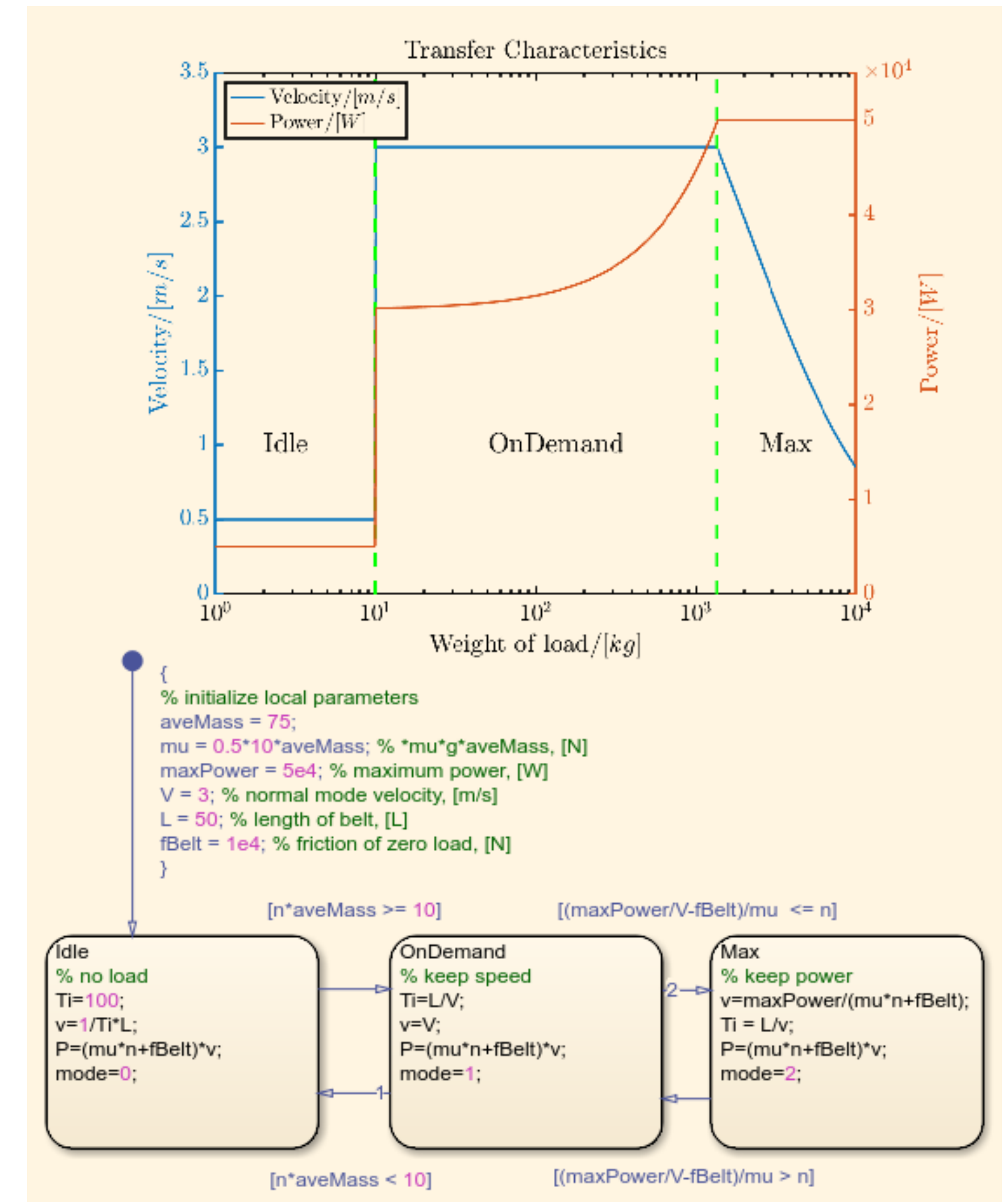
Copyright 2018-2019 The MathWorks, Inc.

Model-in-the-Loop: a Variable Speed Conveyor Belt System in Simulink

The dynamics of the variable speed conveyor belt can be represented using a stateflow diagram.

In the example, the conveyor belt has three modes, which are modeled based on the flow of expected passengers:

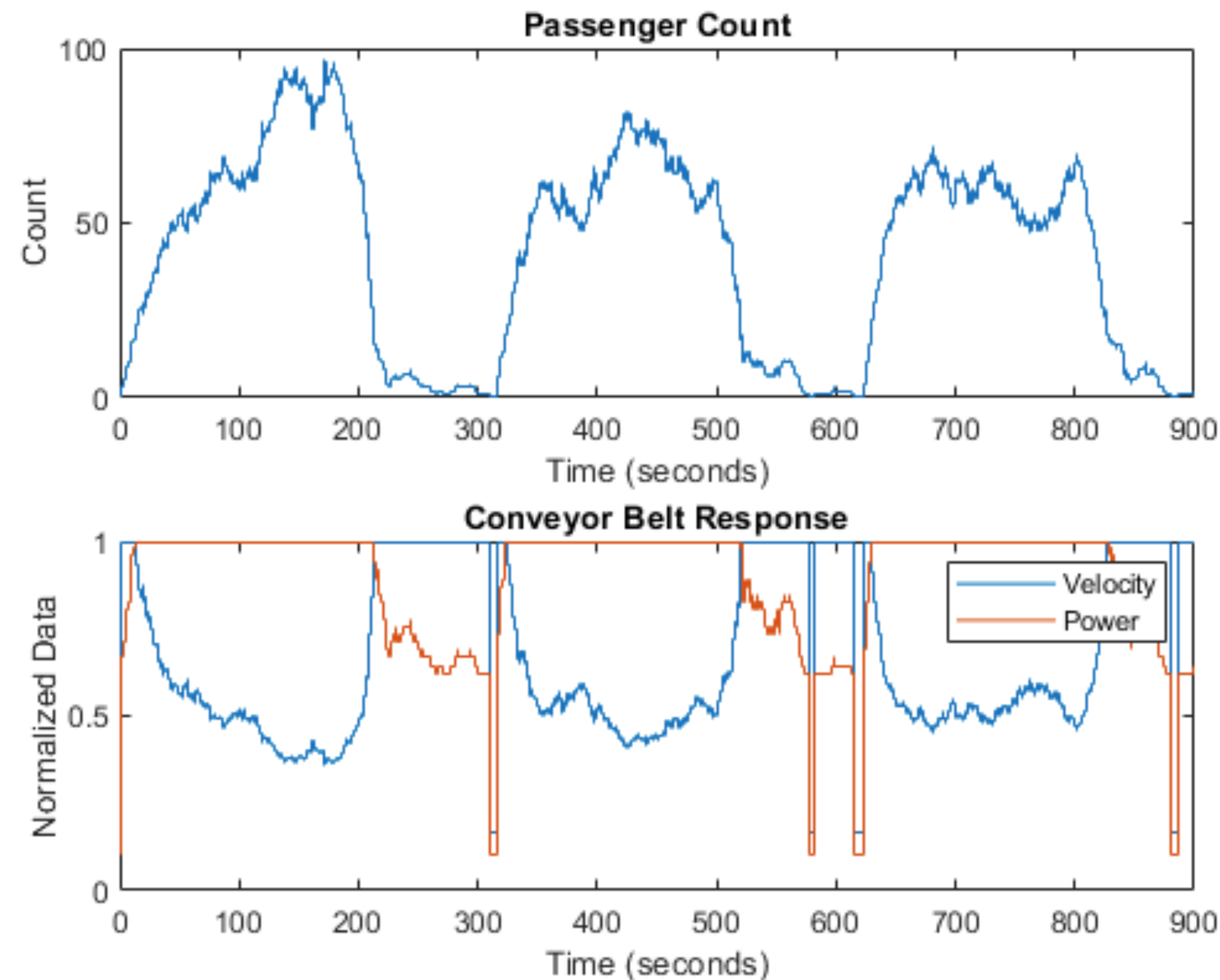
- Idle: the weight of the load is small. The belt runs at a low speed to conserve energy.
- OnDemand: the normal operating mode. It maintains the optimal speed for passenger comfort and throughput.
- Max: maximum power mode. The load weight is too great for the conveyor belt to maintain optimal speed.



Model-in-the-Loop: a Variable Speed Conveyor Belt System in Simulink

Two simulations are combined:

- the first plots the number of passengers versus simulation time,
- the second highlights the response of the conveyor belt to the traffic.



- Three operation cycles are observed within 900 seconds. Each cycle is 300 seconds long, which aligns with the period of the arrival rate.
- The first two-thirds of each period correspond to *rush hour*, when the number of passengers on the conveyor belt increases.
- Consequently, the conveyor belt enters **Max** mode.
- In the last third of each period, the airport is at *normal hours*, followed by a *free hour*.
- The number of passengers on the conveyor belt drops to zero. The conveyor belt then operates in **OnDemand** and **Idle** modes accordingly.

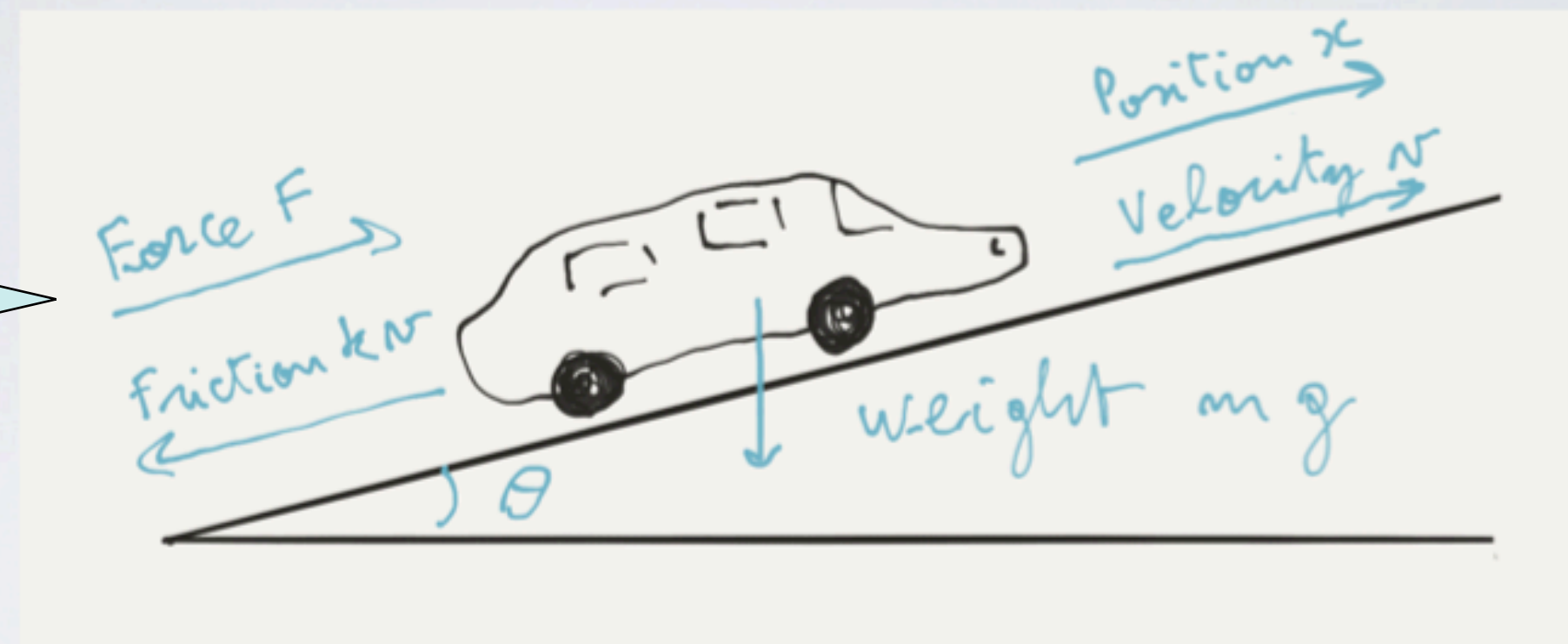
Model-in-the-Loop: a Variable Speed Conveyor Belt System in Simulink

The space of input signals may be **vast and complex to model**.

In the example, there was only one input (the passengers), but what if we work in a more complex scenario?

A cruise control system

The inputs may be influenced by multiple environmental aspects (e.g., wind)



F

θ

$$\text{real } x_L \leq x \leq x_U; v_L \leq v \leq v_U$$
$$\dot{x} = v$$
$$\dot{v} = \frac{(F - kv - mg \sin \theta)}{m}$$

v

Model-in-the-Loop: Challenges to Automatically Test Models

The space of input signals may be **vast and complex to model**.

In the example, there was only one input (the passengers), but what if we work in a more complex scenario?

What types of **requirements/properties** do we check?

In some cases, model execution is **costly**. **Test oracles are not known in advance**; they involve analyzing changes in value over time (e.g., signal patterns) and assessing risk levels.

Simulable environmental/physical models **are often unavailable or inaccurate and imprecise**; for example, they struggle to handle continuous operators. They are based on **model structural coverage**, which has limited fault-detection capabilities and can handle only linear systems and specific properties.

There is **little work on verifying and testing** controllers and decision-making components of Cyber-Physical Systems.

Hence, there are limited chances to test these models accurately...

Model-in-the-Loop: Testing Models through Diversity

A possible strategy is to generate tests that cover diverse situations.

The assumption is that **the more diverse the test cases, the higher their fault-revealing capabilities.**

Defining diversity can be **challenging** and depends on the context and the Cyber-Physical Systems we develop.

By generating random inputs, e.g., using meta**heuristic and evolutionary algorithms**, diversity can be stimulated.

Once the properties to verify (e.g., the coverage of the model's blocks) are defined, fitness functions can be specified, and inputs can be searched for **that optimize the desired property.**

Automated solutions are often better than manual testing because they can generate many inputs that introduce uncertainty and undesired inputs, thereby exercising the models under different conditions.

Software-in-the-Loop: Applying Model-Based Testing

Once a theoretical model describing the Cyber-Physical System is created, developers add **software components to manage the various tasks required by the system**. This step of the development process is the **software-in-the-loop** stage.

Although we do not yet have a complete system to test due to a lack of hardware, **we would like to test whether the software we have developed is sufficient before integrating the hardware**.

As such, **we still need to be abstract and rely on models**.

Unlike simulation models, however, we have software built into the models in this case.

The testing technique most commonly used in this stage is model-based testing, which aims to design and, optionally, *execute artifacts to perform system testing*.

Models can represent the desired behavior of a system under test (SUT), testing strategies, and testing environments.

Model-based testing is instrumental because **the system under test can fully or partially represent the final system**, perfectly fitting the goal of verifying software before including hardware components.

Software-in-the-Loop: Applying Model-Based Testing

There are four main approaches known as model-based testing:

- **Generation of test input data from a domain model.** It is used to provide more knowledge about the space of inputs, but it does not verify the model's behavior— it is mainly done for understandability.
- **Generation of test input data from an environmental model.** It is used to specify the expected usage of the system under test or the operation frequency, but does not specify the expected output; it is mainly used for understandability.
- **Generation of test scripts from abstract tests.** It is used to transform abstract descriptions of test cases (e.g., UML sequence diagrams) into low-level executable test cases, primarily to verify the models.
- **Generation of test cases with oracles from a behavioral model.** It provides fully executable tests with the expected output. To enable this, the model must describe the system under test's expected behavior.

From another perspective, model-based testing automates **the design of black-box tests**.

Software-in-the-Loop: Model-Based Testing Steps

1. Model the system under test and its environment;
2. Generate abstract tests from the model.
3. Concretize the abstract tests to make them executable.
4. Execute the tests on the system under test and assign oracles.
5. Analyze the test results (and take corrective actions).

How do we Model a System and its Interactions?

UML-based Testing

Dynamic models, e.g., sequence, activity, or state-chart diagrams.

Formal Testing

Automata, which illustrate finite-state machines, describing states and state transitions

Context-free grammars, where the model is a language and production rules are the possible states of the model.

Labeled transition systems, which describe the potential behavior of a discrete systems - similarly to automata, they describe states and transitions.

Model-based Testing - Deriving Test Cases from Finite State Machines

From Specification

Maintenance: The Maintenance function records the history of items undergoing maintenance.

Suppose the product is covered by a warranty or maintenance contract. In that case, maintenance can be requested by calling the maintenance toll-free number, through the website, or by bringing the item to a designated maintenance station.

If the maintenance is requested by phone or website and the customer is picked up at the customer's site; otherwise, the customer shall ship the item with a designated carrier.

Multiple choices in the first step...

If the customer-provided maintenance contract number is invalid, the item follows the procedure for items not covered by warranty.

If the product is not covered by a warranty or maintenance contract, maintenance can be requested only by bringing the item to a maintenance station. The maintenance station informs the customer of the estimated repair costs.

Maintenance starts only when the customer accepts the estimate.

If the customer does not accept the estimate, the product is returned to the customer.

Determine the possibilities for the next steps...

Minor problems can be repaired directly at the maintenance station. Suppose the maintenance station cannot solve the problem. In that case, the product is sent to the regional maintenance headquarters (if in the US or EU) or to the main maintenance headquarters (otherwise).

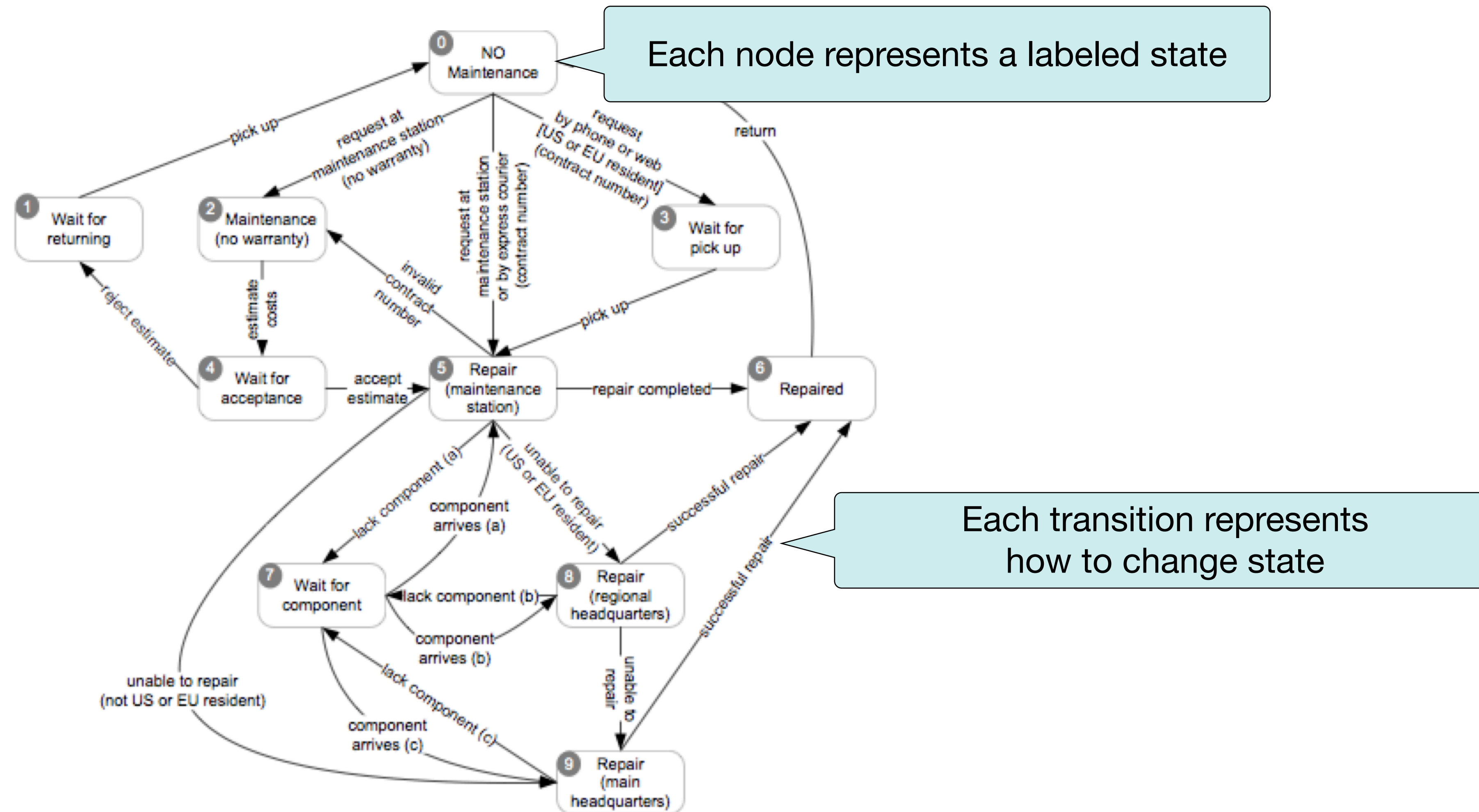
Maintenance is completed when the product is returned to the customer.

And so on...

Once repaired, the product is returned to the customer.

Model-based Testing - Deriving Test Cases from Finite State Machines

...to a finite-state machine...



Model-based Testing - Deriving Test Cases from Finite State Machines

...to an abstract test suite...

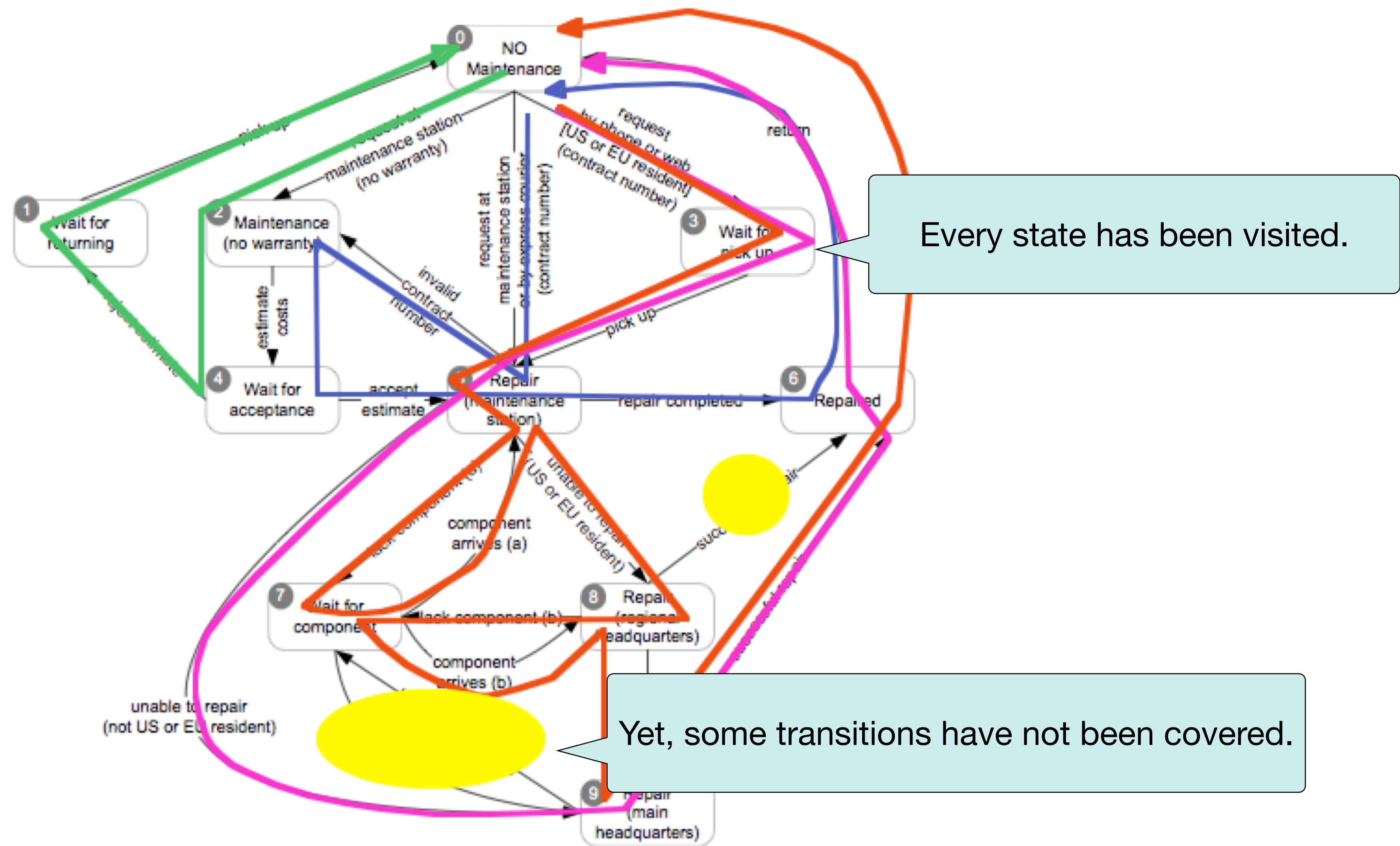
Meaning: From state 0 to state 2 to state 4
to state 1 to state 0

TC1	0	2	4	1	0					
TC2	0	5	2	4	5	6	0			
TC3	0	3	5	9	6	0				
TC4	0	3	5	7	5	8	9	6	0	

- Is this test suite complete? How can we judge its goodness?
- Again, in terms of **coverage**.

Model-based Testing - Deriving Test Cases from Finite State Machines

Assessing Test Goodness



Model-based Testing - Deriving Test Cases from Finite State Machines

Assessing Test Goodness

- **State coverage.** Every state should be visited by at least one test case.
- **Transition coverage.** Every transition between states should be traversed by at least one test case.
Transition coverage is generally preferred to state coverage because a transition can be considered a [precondition, postcondition] pair and, as such, could be more representative when verifying the system under test.
- **Decision structure coverage.** Decisions are similar to branches in source code.
Hence, traditional predicate/clause coverage criteria can be applied.

After defining and assessing the theoretical effectiveness of test cases, they can be concretized with **input data and oracles**.
As soon as enough software is implemented, **tests can be run**.

Hardware-in-the-Loop: Integrating Hardware Components

The model behind the Cyber-Physical System and its software has already been verified at this stage.

The last step is **integrating the hardware components**, i.e., the most challenging part from the testing perspective.

Integration is usually performed through **diagnostic systems** to verify the input/output signals around the Cyber-Physical System. It requires some teamwork among various engineers (not just software engineers).

Testing the integration of hardware components is critical, but **time-consuming and extremely expensive** due to the diverse people and machines involved.

Most effort is spent on modeling, hoping that good models will save effort and money at the hardware-in-the-loop stage.

A close-up photograph of a computer keyboard. The central focus is a large, white, rectangular key with rounded corners. On this key, there is a dark blue icon of a coffee cup with three wavy lines above it representing steam. Below the icon, the word "Break" is printed in a dark blue, serif font. The key is set against a light-colored, textured keyboard surface. Other keys are visible in the background, including one with a double quote symbol and another with a four-way arrow symbol, but they are out of focus.

Break